

Protected While Learning, Unsafe When Released: Reward Poisoning Across Runtime-Assurance Contracts

Abstract—Runtime assurance can keep an adaptive controller safe while hiding an unsafe raw policy. This becomes a security boundary when evidence collected under a resident decision module authorizes permanent release. We show that a bounded attacker who changes only rewards in an online update log can exploit this boundary.

For standardized-advantage policy gradients, we derive the exact normalized reward-influence set of an unclipped batch and reduce positive support against linear release halfspaces to second-order-cone feasibility. Safe-Control-Gym experiments demonstrate delayed raw-policy failures despite protected adaptation. In a locked three-seed cartpole audit, 71 state-snapshot pairs pass the same five-step reverse-switch check. Permanent release fails on 19, across all learner seeds. Repeating that check under resident predictive authority switches on exactly those 19 states and has no observed failure; a one-step resident kernel still fails on 14.

Additional paired audits compare commit, freeze, shielding, and model-predictive safety certification, including coverage and runtime costs. The result is contract-specific: finite reverse-switch evidence can support a resident monitor but cannot by itself certify permanent raw release. The exact analysis excludes clipped batches, the attack does not stably transfer to official PPO, and safe admitted updates do not outperform freeze in our benign-adaptation smoke.

I. INTRODUCTION

Runtime assurance changes what an observer learns about an adaptive controller. If a decision module replaces unsafe proposals with actions from a trusted baseline controller, every adaptation rollout may remain physically safe even while corrupted update data move the raw policy toward failure. This masking is benign while the decision module remains resident: the deployed object is the assured composition. It becomes a security problem when evidence collected under that composition is reused to authorize *permanent raw release*, after which the learned policy acts without the runtime authority that made the evidence safe.

This deployment distinction is easy to blur because runtime-assurance systems already support online upgrades, forward switching, reverse switching, and policy repair. Simplex retains a decision module and assured baseline controller [1]; Neural

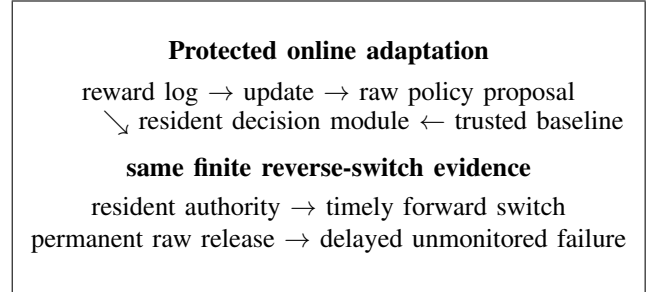


Fig. 1. The evidence contract is the security boundary. A finite prediction can be re-evaluated by a resident decision module, but a one-time pass does not cover the unmonitored future after permanent release.

Simplex adds reverse switching and online retraining [2]; and barrier-based Simplex derives switching conditions that tolerate an unsafe or adversarial advanced controller under bounded model error [3]. Runtime policy repair [4], repair with preservation [5], and safe policy projection [6] further constrain or repair learned policies. These works establish that resident assurance and trusted repair can be effective. They do not establish that finite evidence sufficient for a resident decision module remains sufficient after that module is removed, when the update log itself is adversarial.

We study that gap under a narrow attacker. During protected online adaptation, the attacker changes only scalar rewards written to a policy-gradient update log, within a declared per-step budget. It cannot write actions, gradients, parameters, optimizer state, dynamics, or deployment states. The goal is not to evade the resident action selector. Instead, it steers the raw policy toward a target that passes a finite reverse-switch check at release time but fails later without the decision module. We call this a *certificate-evasive update*: the update is evasive with respect to a stated finite release contract, not a proof that a sound invariant certificate has been defeated.

The attack channel has useful structure. For a fixed policy-gradient batch, bounded reward changes first form a centered reward-to-go zonotope. Because our learners standardize advantages, the resulting parameter update is *not* an affine zonotope: normalization maps the centered returns to the unit sphere before the policy-score map is applied. We derive this normalized reward-influence set exactly when gradient and parameter clipping are inactive. Positive support against a linear certificate halfspace reduces to a second-order-cone feasibility problem with scalar bisection. This gives an attack-

specific bridge from reward corruption to release-gate margin, while making the clipping boundary explicit.

The main experiment compares deployment contracts directly. We lock a five-step reverse-switch check developed on one cartpole learner seed, recover three previously locked poisoned snapshots, and use disjoint evaluation-state pools. Of 72 selected state–snapshot pairs, 71 pass the initial check. Permanent raw release then fails physically on 19/71 pairs, with failures in all 3/3 learner seeds. A resident predictive Simplex mechanism repeats the same five-step check at every physical step, switches exactly on those 19 states four steps before their paired raw failure, and has 0/71 observed failures. Retaining only a one-step action kernel is not enough: it fails on 14/71 pairs and encounters 41 empty-kernel fallbacks. Thus the result is not the overbroad claim that any resident filter fixes the problem; recoverability and continued authority matter.

Broader Safe-Control-Gym evidence establishes the attack and its costs. Reward-log-only poisoning causes 38/188 delayed cartpole failures and 68/72 delayed two-dimensional quadrotor failures, compared with 1/188 and 0/72 for paired clean residual-REINFORCE learners. Commit admission and always-freeze have no observed failure. A unified quadrotor audit compares raw release, commit, freeze, a permanent backup shield, and the official linear MPSC on identical rollout blocks; it exposes different coverage, intervention, reward, and latency costs. Two locked negative gates remain central: the fixed-budget attack does not stably reproduce in the official PPO learner, and admitted updates do not outperform freeze under a benign actuator shift.

Our contributions are:

- We identify a concrete assurance-contract vulnerability: reverse-switch evidence collected under protected adaptation does not transfer automatically to permanent raw release under adversarial online updates. We distinguish this claim explicitly from prior resident Simplex, shielded learning, policy repair, and safe-update work.
- We derive the normalized reward-influence geometry of a standardized-advantage policy-gradient batch and an exact positive-support computation for linear release-gate halfspaces, conditional on inactive gradient and parameter clipping.
- We demonstrate certificate-evasive reward updates and directly compare permanent release, a failed one-step resident kernel, and repeated predictive resident authority. The locked three-seed paired audit yields 19/71, 14/71, and 0/71 physical failures, respectively.
- We retain the broader two-system attack, contract-cost audits, and negative PPO and benign-utility results to delimit learner generality, sampled-certificate strength, and the practical cost of each sound escape condition.

II. MODEL AND PROBLEM STATEMENT

A. Attacked Histories and Plausible States

We consider a discrete-time cyber-physical system

$$x_{t+1} = f(x_t, u_t, w_t), \quad \tilde{y}_t = h(x_t) + a_t, \quad (1)$$

where x_t is the physical state, u_t is the control input, $w_t \in \mathcal{W}$ is ordinary process uncertainty, and a_t is an FDI signal applied to the measurement channel. The controller observes an attacked history

$$h_t = (\tilde{y}_{0:t}, u_{0:t-1}, \tilde{\ell}_{0:t}), \quad (2)$$

where $\tilde{\ell}_t = \ell_t + b_t$ denotes rewards, logs, or auxiliary signals used by the learning rule and b_t is update-data corruption. The two channels need not both be present. Sensor FDI a_t creates uncertainty about the physical state; log corruption b_t can redirect an update even when the state is trusted. Both matter to the certificate lifecycle because the update rule consumes h_t .

Following severe sensor-attack safety work [7], we do not assume that h_t identifies a unique physical state. For an attack family $\mathcal{A}$, define the plausible-state set

$$X_{\mathcal{A}}(h_t) = \{x : \exists E \in \mathcal{A}, H_t(E) = h_t, x_t(E) = x\}. \quad (3)$$

This set is the semantic object over which any strategic-FDI-sound certificate must range. If a certificate is sound only for a nominal estimate $\hat{x}(h_t)$, but $X_{\mathcal{A}}(h_t)$ contains another compatible state, the certificate is not sound against the attack family.

B. Online-Updated Controllers

The controller is a history-dependent policy π_{θ_t} with parameters θ_t . It selects actions as $u_t = \pi_{\theta_t}(h_t)$ and updates according to

$$\theta_{t+1} = U(\theta_t, h_t). \quad (4)$$

We call an update nontrivial if it changes the controller's action map on some reachable or admissible history. Parameter changes that leave all actions unchanged are behaviorally trivial and do not count as learning in this paper.

C. Certificates and Certified Kernels

Let Γ denote a certificate type. In this paper, Γ can be an invariant set, a barrier certificate, a reachability certificate, or a runtime shield. For a history h , the certificate induces a set of actions that are certified for every plausible state:

$$K_{\Gamma}(h) = \{u \in \mathcal{U} : u \text{ satisfies } \Gamma \text{ for all } x \in X_{\mathcal{A}}(h)\}. \quad (5)$$

We call $K_{\Gamma}(h)$ the *certified kernel*. Existing severe-attack safety filters can be viewed as mechanisms for constructing and acting within such kernels for fixed controllers. Our focus is the additional gate required when U changes π_{θ} after deployment.

Definition 1 (Certified online learning). *A mechanism is a certified online learner at history h if it accepts a certificate for the updated controller $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$, and*

claims that the next certified action is sound for every state in $X_{\mathcal{A}}(h)$.

Definition 2 (Learning-freeze frontier). *For a family of histories parameterized by attack strength or ambiguity radius, the learning-freeze frontier is the boundary at which $K_{\Gamma}(h)$ becomes empty, the candidate updated action leaves $K_{\Gamma}(h)$, or the candidate parameter leaves the corresponding certified set $\Theta_{\Gamma}(h)$. Beyond this boundary, a sound certified learner must reject the certificate, freeze the update, or obtain extra trusted state information.*

III. THREAT MODEL

A. System Roles and Deployment Contracts

The defender deploys an online-updated controller π_{θ_t} and a certificate mechanism. A severe-attack-aware estimator or filter can return a plausible-state set $X_{\mathcal{A}}(h_t)$ and certified action kernel $K_{\Gamma}(h_t)$. The defender may also possess a trusted state anchor, backup controller, or external fail-safe, but each is an explicit architectural assumption and cost.

We distinguish two deployment contracts. Under *snapshot deployment*, a safety layer protects data collection and adaptation, after which a raw snapshot $\pi_{\theta+}$ may be committed for deployment. The snapshot itself therefore requires admission. Under *permanently filtered deployment*, a sound filter F remains online and the certified object is the composition $F \circ \pi_{\theta+}$. Our raw-snapshot attack does not bypass such a filter; we evaluate permanent filtering as an escape condition and measure its intervention and reward cost.

B. Attack Channels and Capabilities

The general history model permits two attack channels. Observation FDI a_t changes the state semantics of the history and enlarges $X_{\mathcal{A}}(h_t)$. Update-data corruption b_t changes rewards, logs, or auxiliary learning signals and can redirect $U(\theta_t, h_t)$ even when the state is trusted. The formal lifecycle condition applies to either channel because both can affect the next action map.

The end-to-end learner experiments deliberately use the narrower reward-log channel. The attacker is white-box and batch-aware but modifies only scalar rewards stored for an ordinary policy-gradient update, within a declared per-step L_{∞} budget. It cannot write actuator commands, physical states, observations, dynamics, gradients, policy parameters, optimizer state, initial deployment states, or certificate outcomes. The adaptation filter sees and acts on the learner's proposed action before the plant step. Thus physical safety during data collection does not arise from giving the attacker direct policy-write access.

C. Security and Availability Goals

We separate four attacker or failure objectives:

- an *unsafe certified action*, where the mechanism accepts an applied input that is unsafe for some state in $X_{\mathcal{A}}(h_t)$;
- a *stale certified policy*, where the current applied input is filtered but the accepted raw updated action map leaves the kernel;

- *uncertified learning*, where a nontrivial update continues after certificate rejection; and
- *forced freeze*, where the attacker expands the plausible set or redirects the update until sound learning must stop or re-anchor.

The defender additionally cares about availability: admitted state coverage, accepted update fraction, task reward, filter intervention and rejection, certificate latency, and trusted-anchor use. Zero physical violations with no benefit over always-freeze is safe but does not establish useful learning.

D. Non-goals

We do not assume that every attack makes safety impossible. Existing severe-attack filters, secure estimators, and fault-tolerant CBF methods can preserve safety when their assumptions make $K_{\Gamma}(h_t)$ nonempty [7]–[9]. We do not claim to defeat a permanently sound runtime filter, to prove continuous-state invariance from a sampled certificate, or to establish vulnerability for all learner families. The question is narrower: which certificate must cover an update, which escape conditions remain sound, and what coverage and availability costs follow?

IV. CERTIFICATE LIFECYCLE ATTACKS

This section separates three failure modes that are often conflated.

Nominal-state certification. A nominal certificate checks safety only at an estimate $\hat{x}(h)$ or at the reported sensor value. Under FDI, this can accept an action that is safe for $\hat{x}(h)$ but unsafe for another state in $X_{\mathcal{A}}(h)$. This is the classical state-semantics failure already addressed by severe-attack safety filters.

Action-only filtering. A robust safety filter can repair the current action by projecting it into $K_{\Gamma}(h)$. This protects the physical action applied at that step when the kernel is nonempty. It does not, by itself, certify the learner update. If the update rule still changes θ_t to θ_{t+1} such that the raw policy $\pi_{\theta_{t+1}}$ leaves $K_{\Gamma}(h)$, then the certificate applies to the filtered action, not to the updated learner. The next certificate must therefore reason about the updated action map, not only about the one action that was just projected.

Uncertified learning after rejection. When $K_{\Gamma}(h) = \emptyset$, a severe-attack filter can reject or enter fail-safe mode. If the learner nevertheless updates θ_t using the same attacked history, this update is not certified learning. The mechanism may still operate as an engineering fail-safe, but it cannot claim that the online learner remained inside a sound certificate lifecycle.

Protected adaptation, unsafe deployment. An adaptation shield can keep every data-collection transition safe while an attacker corrupts only the log consumed by the learner. If the resulting snapshot is later deployed without the shield, the attack succeeds only through the legitimate update rule. This delayed failure is distinct from evading the runtime shield. A sound snapshot lifecycle therefore requires a commit certificate before the protected adaptation environment releases a raw policy.

These attacks motivate separate metrics. An *unsafe certified action* counts accepted certificates whose applied action is unsafe for some plausible state. A *stale certified policy* counts accepted certificates for which the updated raw policy leaves the kernel. *Uncertified learning* counts nontrivial updates after certificate rejection. End-to-end studies add delayed deployment violations, admission coverage, paired clean/poison outcomes, update fraction, intervention, reward, and certificate runtime.

V. THE CERTIFICATE LIFECYCLE GATE

A. Current- and Future-History Gates

The central condition has two levels. The first is a current-history condition: after an update, the action selected at the history used for the update must still be certified. The second is a lifecycle condition: the updated action map must remain certified over the future attacked histories that can be reached after accepting the update.

Lemma 1 (Current-history gate). *Let $M = (\pi_\theta, U, \text{Cert})$ be a history-dependent online controller. For an attacked history h , let $X_{\mathcal{A}}(h)$ be the plausible-state set and let $K_\Gamma(h)$ be the certified kernel induced by certificate type Γ . If M accepts a sound certificate for the updated controller $\pi_{\theta_{t+1}}$, where $\theta_{t+1} = U(\theta_t, h)$, then*

$$\pi_{\theta_{t+1}}(h) \in K_\Gamma(h). \quad (6)$$

Consequently, if $K_\Gamma(h) = \emptyset$, then M cannot both accept the certificate and allow a nontrivial online update at h .

Proof. Soundness of the certificate means that the certified action must satisfy the certificate condition for every physical state that is compatible with the attacked history and the attack model. By definition, that set of states is $X_{\mathcal{A}}(h)$, and the set of actions satisfying the certificate condition for all such states is $K_\Gamma(h)$. If the updated controller remains in certified operation after the update, its selected action at h is $\pi_{\theta_{t+1}}(h)$ and must therefore lie in $K_\Gamma(h)$. If $K_\Gamma(h) = \emptyset$, no such certified action exists. A sound mechanism must then reject the certificate, freeze or roll back the update, or use additional trusted information that changes the plausible-state set and hence the kernel. $\square$

Proposition 1 (Action filtering is not update certification). *Let $F(v, h)$ be an action filter that returns an applied input in $K_\Gamma(h)$ whenever the kernel is nonempty. Consider a mechanism that first computes a candidate update $\theta^+ = U(\theta_t, h)$, then applies*

$$u_t = F(\pi_{\theta^+}(h), h), \quad (7)$$

but accepts the update without constraining the raw updated policy π_{θ^+} . If $\pi_{\theta^+}(h) \notin K_\Gamma(h)$, then the applied input can be certified while the updated learner is not certified at h . Consequently, action-only filtering does not imply the current-history gate in Lemma 1.

Proof. The filter property gives $u_t \in K_\Gamma(h)$, so the actuator input applied at the current step can satisfy the certificate.

Lemma 1, however, is a condition on the updated controller claimed to remain in certified operation: it requires $\pi_{\theta^+}(h) \in K_\Gamma(h)$. When $\pi_{\theta^+}(h) \notin K_\Gamma(h)$, the certificate covers the filtered actuator channel, not the raw action map produced by the updated learner. Accepting the learner update under the same certificate therefore creates a stale certified policy. $\square$

The lemma and proposition are local gates. They are enough to expose the stale-policy failure in a one-step study, but a controller whose parameters define actions at many future histories needs a stronger object. Let $\mathcal{H}_{\mathcal{A}}(h, \theta^+)$ be the set of attacked histories that can arise after history h when the updated controller π_{θ^+} is used and the attacker remains inside $\mathcal{A}$. Define the certified parameter set

$$\Theta_\Gamma(h) = \{\theta : \forall h' \in \mathcal{H}_{\mathcal{A}}(h, \theta), \pi_\theta(h') \in K_\Gamma(h')\}. \quad (8)$$

Theorem 1 (Future-history lifecycle gate). *Let $\theta^+ = U(\theta_t, h)$ be a candidate online update. If M accepts a sound lifecycle certificate for π_{θ^+} from history h over the attack family $\mathcal{A}$, then*

$$\theta^+ \in \Theta_\Gamma(h), \quad (9)$$

equivalently,

$$\forall h' \in \mathcal{H}_{\mathcal{A}}(h, \theta^+), \pi_{\theta^+}(h') \in K_\Gamma(h'). \quad (10)$$

If there exists $h' \in \mathcal{H}_{\mathcal{A}}(h, \theta^+)$ such that $\pi_{\theta^+}(h') \notin K_\Gamma(h')$, then accepting the update creates a stale certified policy. If some reachable h' has $K_\Gamma(h') = \emptyset$, certified learning cannot continue through that history without rejecting, freezing, rolling back, or obtaining trusted information that changes $X_{\mathcal{A}}(h')$.

Proof. A lifecycle certificate claims sound certified operation not only at the update history h , but at every attacked history h' reachable under the updated controller and the admissible attack family. Applying the soundness argument from Lemma 1 pointwise to each such h' gives $\pi_{\theta^+}(h') \in K_\Gamma(h')$. This is exactly the definition of $\theta^+ \in \Theta_\Gamma(h)$. If membership fails for any reachable history, the certificate no longer certifies the updated action map there; if the kernel is empty, no history-dependent action can satisfy the certificate at that history without changing the information set or rejecting certified operation. $\square$

Proposition 2 (Linear-policy parameter gate). *Consider a finite abstraction of future attacked histories indexed by observations $z \in \mathcal{Z}$, and a linear policy $\pi_\theta(z) = \theta^\top \phi(z)$. If each future-history certificate produces an interval kernel $K_\Gamma(z) = [\ell_z, u_z]$, then the lifecycle condition over $\mathcal{Z}$ is equivalent to the halfspace system*

$$\theta^\top \phi(z) \leq u_z, \quad -\theta^\top \phi(z) \leq -\ell_z, \quad \forall z \in \mathcal{Z}. \quad (11)$$

With optional box constraints on θ , the certified update set is a convex polytope. Rejecting candidates outside this set or projecting them onto it is sound for the finite abstraction.

Proof. For a fixed z , membership $\pi_\theta(z) \in [\ell_z, u_z]$ is exactly the pair of linear inequalities above. Enforcing this pair for

every $z \in \mathcal{Z}$ is therefore equivalent to enforcing the future-history lifecycle gate on the abstraction. Intersecting these halfspaces with parameter box constraints preserves convexity. A candidate accepted after a successful feasibility check or projection satisfies every inequality, so its induced action lies in every certified interval kernel in the abstraction. $\square$

B. Normalized Reward Influence on a Release Gate

We now connect the reward-log attacker to the halfspaces above. Consider a fixed on-policy batch of length T . Let $S \in \mathbb{R}^{T \times d}$ contain the policy-score row vectors, let $r \in \mathbb{R}^T$ be the logged rewards, and let $H_{ij} = \mathbf{1}[j \geq i]$ be the reward-to-go operator. Define the centering matrix

$$C = I - \frac{1}{T} \mathbf{1} \mathbf{1}^\top \quad (12)$$

and let the attacker choose $\delta \in [-\epsilon, \epsilon]^T$. The centered return vector is

$$y(\delta) = CH(r + \delta). \quad (13)$$

The fixed-batch assumption is appropriate for a reward-log-only attacker: changing a reward after collection does not change the states, actions, or score rows already in the batch.

Proposition 3 (Standardized reward-influence set). *Suppose the learner standardizes the centered reward-to-go advantages with population standard deviation, $\|y(\delta)\|_2 > 0$ throughout the reward box, and neither gradient-norm clipping nor parameter clipping is active. A gradient-ascent step of size α is*

$$\theta^+(\delta) = \theta + \frac{\alpha}{\sqrt{T}} S^\top \frac{y(\delta)}{\|y(\delta)\|_2}. \quad (14)$$

Consequently, the exact one-batch update set is the image of the centered reward-to-go zonotope under normalization and the policy-score map; in general, it is not a parameter-space zonotope.

Proof. The standardized advantage vector is $y/\sqrt{T^{-1}\|y\|_2^2} = \sqrt{T}y/\|y\|_2$. Substitution into the usual mean score-function gradient $T^{-1}S^\top(\sqrt{T}y/\|y\|_2)$ gives (14). Equation (13) is affine in the reward-box variable, but division by its Euclidean norm is not; hence the normalized image is generally non-affine. $\square$

Theorem 2 (Positive support against a release halfspace). *Under Proposition 3, consider a release gate $a^\top P\theta \leq b$, where P selects or transforms policy parameters, and define*

$$q = \frac{\alpha}{\sqrt{T}} SP^\top a. \quad (15)$$

For any fixed $t \geq 0$, a reward perturbation whose gate-coordinate increment is at least t exists if and only if the following second-order-cone feasibility problem is feasible:

$$\begin{aligned} -\epsilon \mathbf{1} &\leq \delta \leq \epsilon \mathbf{1}, \\ t\|CH(r + \delta)\|_2 &\leq q^\top CH(r + \delta). \end{aligned} \quad (16)$$

Feasibility is monotone in t , so bisection on $[0, \|q\|_2]$ computes the exact positive support

$$\max \left\{ 0, \sup_{\|\delta\|_\infty \leq \epsilon} \frac{q^\top y(\delta)}{\|y(\delta)\|_2} \right\}. \quad (17)$$

If this support does not exceed the current halfspace margin $b - a^\top P\theta$, no admissible reward perturbation can cross that halfspace in this batch.

Proof. By (14), the change in the halfspace left-hand side is $q^\top y(\delta)/\|y(\delta)\|_2$. Because the denominator is positive, requiring this ratio to be at least $t \geq 0$ is equivalent to (16). For fixed t , a norm bounded above by an affine expression is a second-order-cone constraint. Any witness feasible at t remains feasible at a smaller nonnegative value, and Cauchy-Schwarz bounds the ratio by $\|q\|_2$, establishing the bisection interval. The margin statement follows by adding the maximum possible increment to the current gate coordinate. $\square$

This theorem is deliberately batch-local. Later batches change S , r , and the visited states; accumulated attacks compose state-dependent normalized updates rather than one fixed convex set. If gradient clipping is active, the update is additionally radially rescaled; if parameter clipping is active, it is projected coordinatewise. We do not claim Theorem 2 is exact across either branch. Section VII validates the unclipped formula on a real batch and treats clipped learners empirically.

C. A Margin-Robust Finite-Abstraction Lift

Proposition 2 is exact on its listed grid. Lifting that result to continuous reachable histories requires coverage and regularity assumptions. Let $z = \psi(h')$ be a finite-dimensional representation on which the updated policy depends. Suppose the fixed-controller certificate can be written as finitely many obligations

$$c_j(x, u, w, x^+) \leq 0, \quad j = 1, \dots, m, \quad (18)$$

where $x^+ = f(x, u, w)$. Invariant, discrete-time barrier, and bounded-horizon reachability checks can be written in this form after augmenting the state with time or certificate memory.

Theorem 3 (Margin-robust finite-abstraction sufficiency). *Fix a candidate snapshot π_θ and a future-history horizon. Let $\{(z_i, x_{iq}, w_{ir})\}$ be a finite set such that, for every reachable attacked history h' , every $x \in X_A(h')$, and every $w \in \mathcal{W}$, some sampled triple satisfies*

$$\|z - z_i\| \leq \delta_z, \quad \|x - x_{iq}\| \leq \delta_x, \quad \|w - w_{ir}\| \leq \delta_w, \quad (19)$$

where $z = \psi(h')$. Assume:

- 1) $\pi_\theta(z) \in \mathcal{U}$ and $\|\pi_\theta(z) - \pi_\theta(z')\| \leq L_\pi \|z - z'\|$;
- 2) the certificate model $\hat{f}$ has uniform error $\|f(x, u, w) - \hat{f}(x, u, w)\| \leq \varepsilon_f$;
- 3) f is separately Lipschitz with constants L_f^x, L_f^u, L_f^w ; and
- 4) each c_j is separately Lipschitz in (x, u, w, x^+) with constants $L_j^x, L_j^u, L_j^w, L_j^+$.

Define

$$\begin{aligned}\eta_j = & (L_j^x + L_j^+ L_f^x) \delta_x \\ & + (L_j^u + L_j^+ L_f^u) L_\pi \delta_z \\ & + (L_j^w + L_j^+ L_f^w) \delta_w + L_j^+ \varepsilon_f.\end{aligned}\quad (20)$$

If every sampled triple satisfies

$$c_j(x_{iq}, \pi_\theta(z_i), w_{ir}, \hat{f}(x_{iq}, \pi_\theta(z_i), w_{ir})) \leq -\eta_j \quad (21)$$

for all j , then $\pi_\theta(\psi(h')) \in K_\Gamma(h')$ for every reachable attacked history h' in the horizon. Hence the snapshot satisfies the continuous future-history lifecycle condition of Theorem 1.

Proof. Fix h' , $x \in X_A(h')$, and $w \in \mathcal{W}$, and select a sampled triple using (19). Write $u = \pi_\theta(z)$ and $u_i = \pi_\theta(z_i)$. Policy regularity gives $\|u - u_i\| \leq L_\pi \delta_z$. The model-error and Lipschitz assumptions give

$$\begin{aligned}\|f(x, u, w) - \hat{f}(x_{iq}, u_i, w_{ir})\| \leq & \varepsilon_f + L_f^x \delta_x \\ & + L_f^u L_\pi \delta_z + L_f^w \delta_w.\end{aligned}\quad (22)$$

Applying the four Lipschitz bounds of c_j and collecting terms yields

$$\begin{aligned}c_j(x, u, w, f(x, u, w)) \leq & c_j(x_{iq}, u_i, w_{ir}, \\ & \hat{f}(x_{iq}, u_i, w_{ir})) + \eta_j \\ \leq & 0.\end{aligned}\quad (23)$$

The argument holds for every plausible state, disturbance, certificate obligation, and reachable attacked history, which is precisely membership in the corresponding kernels over the stated horizon. $\square$

Theorem 3 is a conditional verification result, not a reachable-set construction. Its joint-cover premise must itself be justified; a finite rollout sample does not establish it. The coverage audit in Section VII empirically probes this missing premise but does not replace a cover proof.

Proposition 4 (Plausible-set and freeze-frontier monotonicity). *Fix a history domain $\mathcal{H}$, dynamics, certificate obligations, and policy class. If two information contracts satisfy $X_1(h) \subseteq X_2(h)$ for every $h \in \mathcal{H}$, let $K_{\Gamma,k}$ be the kernel induced by X_k and let $\Theta_{\Gamma,k} = \{\theta : \pi_\theta(h) \in K_{\Gamma,k}(h), \forall h \in \mathcal{H}\}$. Then*

$$K_{\Gamma,2}(h) \subseteq K_{\Gamma,1}(h) \quad \text{and} \quad \Theta_{\Gamma,2} \subseteq \Theta_{\Gamma,1}. \quad (24)$$

Consequently, for a nested ambiguity family $X_{\rho_1}(h) \subseteq X_{\rho_2}(h)$ whenever $\rho_1 \leq \rho_2$, certified update feasibility is non-increasing in ρ : once the certified parameter set is empty, it remains empty at larger ambiguity. Conversely, a trusted anchor that pointwise shrinks the plausible-state sets can only weakly enlarge the kernels and certified parameter set.

Proof. An action certified for every state in the larger set $X_2(h)$ is certified for every state in its subset $X_1(h)$, proving kernel inclusion. Requiring a policy action to lie in the smaller kernel $K_{\Gamma,2}(h)$ for every $h \in \mathcal{H}$ is at least as restrictive as requiring membership in $K_{\Gamma,1}(h)$, proving parameter-set inclusion. The frontier and anchor statements follow by applying these inclusions to nested families. $\square$

The monotonicity comparison holds on a fixed history domain and certificate contract. If an anchor changes which histories are reachable, or if two sampled plausible sets are not nested, neither conclusion follows automatically. The P3 anchor audit therefore constructs nested samples explicitly.

The future-history theorem is a necessary certificate condition; Proposition 2 is exact only on its abstraction; and Theorem 3 supplies a sufficient lift only under its stated cover, regularity, and margin assumptions. None is a complete controller synthesis procedure or a claim that safety under sensor attacks is impossible.

D. Instantiating the Kernel

For an invariant certificate $I \subseteq S$, the kernel is

$$K_I(h) = \{u \in \mathcal{U} : \forall x \in X_A(h), \forall w \in \mathcal{W}, f(x, u, w) \in I\}. \quad (25)$$

For a barrier certificate $B(x) \geq 0$ with safe set $C = \{x : B(x) \geq 0\}$, the corresponding kernel is

$$K_B(h) = \{u \in \mathcal{U} : \forall x \in X_A(h), \forall w \in \mathcal{W}, B(f(x, u, w)) \geq 0\}. \quad (26)$$

For a reachability certificate, the initial set of the reachability computation must be $X_A(h)$ rather than a nominal state estimate. For a runtime shield, the shield output must lie in $K_\Gamma(h)$; if the kernel is empty, the shield can only reject, enter fail-safe mode, or request a trusted state anchor.

E. A Minimal Counterexample

Consider

$$x_{t+1} = \lambda x_t + u_t, \quad |u_t| \leq u_{\max}, \quad S = [-1, 1]. \quad (27)$$

Suppose FDI makes the same history compatible with $x_t = r$ and $x_t = -r$, where $r \in (0, 1]$ and $\delta = \lambda r$. The one-step safe action set for a state x is

$$\mathcal{U}_{\text{safe}}(x) = [-u_{\max}, u_{\max}] \cap [-1 - \lambda x, 1 - \lambda x]. \quad (28)$$

For $x = r$ and $x = -r$, these sets are individually nonempty but disjoint whenever

$$1 < \lambda r < 1 + u_{\max}. \quad (29)$$

The strict upper bound ensures that each state is individually controllable if it is known. The failure comes from FDI-induced indistinguishability: no single history-dependent action can certify both states.

This example distinguishes fixed filtering from certified learning. A fixed severe-attack filter can reject because the common kernel is empty. A certified online learner must do more: it must also prevent an update from continuing under a certificate that no longer has a feasible kernel.

F. Certificate-Gated Update

The theorem suggests the following update rule. First compute or obtain $X_{\mathcal{A}}(h)$ using a severe-attack-aware estimator or safety filter. Then compute $K_{\Gamma}(h)$. If the kernel is empty, reject the certificate and freeze the update. If the kernel is nonempty, apply the candidate update only if the updated action remains inside the kernel; otherwise project, constrain, or reject the update.

This rule should not be read as a complete controller synthesis method. Its role is to separate the fixed-controller safety problem, which prior work addresses, from the lifecycle condition needed when a controller learns from attacked histories.

VI. LIFECYCLEGATE DESIGN

LifecycleGate is a layer around existing severe-attack-aware filters. It does not construct a new CBF or reachability certificate. Instead, it uses the kernel produced by such a certificate to decide whether the learning update can be accepted.

Interface. At history h_t , the gate calls an estimator to construct $X_{\mathcal{A}}(h_t)$ and calls a certificate module to construct $K_{\Gamma}(h_t)$. It then computes the candidate update $\theta^+ = U(\theta_t, h_t)$. If $K_{\Gamma}(h_t)$ is empty, the gate rejects the certificate and freezes or rolls back the update. If the kernel is nonempty, the gate checks whether the candidate updated action remains inside the kernel. When a future-history abstraction is available, the gate instead constructs $\Theta_{\Gamma}(h_t)$ and checks or projects the full parameter update. A practical implementation can accept the update, project it to a certified update set, reject it, or request a trusted state anchor that shrinks $X_{\mathcal{A}}(h_t)$.

Current-history gate. The minimal gate checks

$$\pi_{\theta^+}(h_t) \in K_{\Gamma}(h_t). \quad (30)$$

This condition is enough to prevent the stale-policy failure at the current history. It is the condition used in the action-level frontier studies.

Lifecycle gate. A stronger gate must reason about future attacked histories reachable after the update. Let $\mathcal{H}_{\mathcal{A}}(h_t, \theta^+)$ denote the set of histories that can arise from h_t under the attack family and the updated controller. A lifecycle certificate requires

$$\forall h' \in \mathcal{H}_{\mathcal{A}}(h_t, \theta^+), \quad \pi_{\theta^+}(h') \in K_{\Gamma}(h'). \quad (31)$$

This is the condition that separates certificate lifecycle preservation from single-step action filtering.

Parameter gate. For linear policies over a finite future-history grid, $\Theta_{\Gamma}(h_t)$ becomes the halfspace system in Proposition 2. The gate can therefore implement certified learning as a small convex feasibility or projection problem over policy parameters rather than as a post-hoc projection of only the current action.

Robust abstraction margin. A grid check is a certificate for its grid unless the implementation also establishes the joint cover and error bounds in Theorem 3. When those quantities are available, the gate tightens each sampled certificate obligation by η_j in (20) before accepting the update. If the tightened

set is empty, the fail-closed response is the same as for an empty kernel. An empirical guard selected from a validation grid is not a substitute for the theorem’s uniform model-error and cover premises.

Design choices. The least conservative design freezes whenever the candidate update leaves the certified set. A less conservative design projects the action or parameter update onto the certified set. An anchor-assisted design invokes trusted sensing only when the kernel is empty or too narrow. These choices trade off learning progress, task cost, and trusted infrastructure.

Snapshot commit versus permanent filtering. For snapshot deployment, LifecycleGate certifies a candidate snapshot on a declared future-history/state set and backtracks the update along the connected segment from the last certified snapshot. Connected backtracking prevents a disconnected safe island from being accepted after an intervening unsafe fraction. For permanently filtered deployment, the certificate must instead cover $F \circ \pi_{\theta^+}$ at runtime. These contracts are not interchangeable: a commit gate exposes certificate coverage and model-mismatch risk, whereas a permanent filter exposes intervention, rejection, and availability cost.

Fail-closed semantics. An empty plausible-state kernel, an uncertified current snapshot, model failure, or solver failure cannot silently inherit the previous certificate. The gate freezes or rolls back and reports the failure. A trusted anchor may resume learning only by changing the justified plausible-state set; it is not modeled as a free oracle.

Utility is an independent property. The gate enforces certificate membership, not improvement of the task objective. A safe update can be neutral or harmful. Consequently, accepted parameter distance is an availability diagnostic but not proof of learning utility; utility requires paired task outcomes against always-freeze under a benign change.

VII. EVALUATION

We ask four questions on Safe-Control-Gym [10]: (Q1) can corruption limited to reward logs cause delayed physical failure; (Q2) how does a bounded reward box influence release-gate halfspaces; (Q3) does the same finite reverse-switch evidence support permanent release and resident predictive authority; and (Q4) which escape contracts prevent failure, at what cost, and with which generality limits?

A. Supplementary Semantic Checks

Before the physical studies, one-dimensional, linear-tank, nonlinear-tank, and cartpole sanity checks instantiate the certificate semantics in Sections V and VI. Across nominal, action-only, and parameter-level gates, they reproduce the expected boundary: an action filter can keep the applied input safe while the raw update leaves the finite certified parameter set; when the common kernel becomes empty, a fail-closed gate freezes. The full ambiguity frontiers, adaptive-search stress test, CBF reference, figures, and row-level outputs remain in the artifact supplement. They are semantic checks, not evidence for the main novelty claim.

B. Reward-Log Poisoning of Online Learners

The preceding experiments expose certificate semantics with controlled update directions. We next test whether the same lifecycle failure arises when updates are produced by an actual learner. The policy is a Gaussian residual actor on top of a trusted controller. Standard reward-to-go REINFORCE gradients are computed from PyBullet transitions. The attacker can change only the scalar reward written to the update log, subject to a per-step L_∞ budget; it cannot write the action, gradient, parameter, optimizer state, dynamics, or deployment state. A backup action filter protects the adaptation trajectory. Deployment then audits the raw learned snapshot, which is the object requiring lifecycle admission.

Training, certificate, calibration, and deployment states are separated by seed. All mechanisms within a learner seed use paired state sets. Candidate attack directions and hyperparameters are fixed on a development seed before the three-seed audits. The exact tests below operate at the paired deployment-state-block level, conditional on the three learned snapshots. Because blocks sharing a learner seed are not independent learner replicates, the p -values summarize within-snapshot discordance; replication across learners is conveyed separately by the consistency over 3/3 seeds. Wilson and paired-block bootstrap intervals reported below have the same conditional scope and are not confidence statements over a population of learner seeds. Table I summarizes the resulting physical deployments.

On cartpole, the clean learner violates 1/188 admitted rollouts and the poisoned action-only learner violates 38/188. There are 37 paired poison-only failures and no clean-only failures (two-sided exact paired $p = 1.455 \times 10^{-11}$). Both commit admission and always-freeze have zero observed violations.

The two-dimensional quadrotor result is stronger. Clean learning has 0/72 violations, while reward-only poisoning causes 68/72, all delayed, across 3/3 learner seeds. The paired discordance is 68 poison-only to zero clean-only (two-sided exact $p = 6.776 \times 10^{-21}$). Commit admission and always-freeze again have zero observed violations. The gate retains nonzero fractions 0.55, 0.40, and 0.70 of the three poisoned pending updates. These fractions establish update availability under attack, not benign learning utility.

C. Release-Gate Geometry and Contract Audit

We first validate the reward-influence analysis on a real, unclipped cartpole batch from learner seed 2040. Reconstructing the implemented gradient from Equation (14) has error 2.78×10^{-17} . The clean gradient norm is 0.196789, neither gradient nor parameter clipping is active, and the minimum centered-return norm over the reward box is 7.993864. Across 22 release-gate halfspaces, the largest discrepancy between an SOCP witness and its computed support is 1.07×10^{-9} . The minimum robust margin is 4.598004, so no halfspace can be crossed in this batch. This negative result matters: the physical attack accumulates changing batch-local influences rather than fabricating a one-step gate crossing. All locked

quadrotor batches reach the gradient cap, so the exact theorem is not applied to them.

We next audit whether finite reverse-switch evidence transfers between deployment contracts. A retrospective 3×3 target-grid analysis requires five-step acceptance on all 42 development states, then ranks accepted targets by 120-step failure. It selects the same target that had already been locked for the original reward-poisoning run. The learned snapshot passes the five-step check on 42/42 states but fails on 13/42 under both the CasADi certificate model and physical PyBullet deployment. Because the original attack did not optimize this newly written constrained objective, this is mechanism validation, not a prospectively designed gate-aware attack.

For the confirmatory contract audit, we freeze that five-step horizon, recover the locked poisoned snapshots for learner seeds 2040–2042, and generate disjoint evaluation pools with seeds 8040–8042. From 32 baseline-admitted candidates per learner, 24 order-spanning states are selected without per-seed tuning. One pair is rejected by the initial five-step check, leaving 71 accepted pairs. Permanent release and resident mechanisms start from identical state-snapshot keys and are evaluated for 120 physical steps.

Table II gives the direct separation. Raw release fails on 6/24, 3/23, and 10/24 accepted pairs in the three learner seeds. The predictive resident mechanism repeats the same five-step raw-policy prediction at every physical step and switches on exactly those 19 paired states. Every switch precedes its raw-release failure by four steps, and no physical failure remains. The two learner seeds not used to develop the horizon account for 13 raw failures and zero predictive-resident failures. In contrast, a resident one-step singleton-state action kernel still fails on 14 pairs and reaches an empty kernel 41 times. We therefore claim neither that a finite-horizon pass proves invariance nor that any resident filter is sufficient. The supported result is narrower: the same finite prediction is effective as a repeatedly evaluated recoverability trigger, but unsound as a one-time authorization for an unmonitored future.

D. Runtime Escape Conditions and Their Cost

Snapshot admission is not the only sound architectural response. If a runtime filter is permanently present and its certificate covers the composed policy $F \circ \pi$, raw-policy failure need not become physical failure. A prior audit established feasibility, but used only 12 rollout blocks rather than the 72 blocks used for the raw and commit mechanisms. We therefore lock a new result-independent pool of 144 states from three disjoint seeds. For each learner seed, the common operating set contains exactly the states certified for both freeze and that seed’s committed snapshot at horizon 100 and guard 0.003. We report its coverage, select eight order-spanning states, and execute all six mechanisms from identical state/reset pairs. Thus every pooled mechanism has 24 paired rollouts and 2400 possible steps.

Table III and Fig. 2 show the resulting separation. The poisoned raw snapshots violate 23/24 rollouts and complete only one. Clean, commit, freeze, the permanent five-step

TABLE I

PAIRED REWARD-LOG POISONING AND SNAPSHOT ADMISSION. “COMMIT” DENOTES FINITE-HORIZON ADMISSION; PERMANENT MECHANISMS USE THE COMMON-STATE AUDIT IN FIG. 2. RAW-SNAPSHOT REWARDS AFTER FAILURE ARE DESCRIPTIVE, NOT UTILITY COMPARISONS.

System	Mechanism	Learner seeds	Rollouts	Violations	Mean reward
Cartpole	Clean REINFORCE	3	188	1	−0.197
Cartpole	Poisoned action-only	3	188	38	−1.269
Cartpole	Poisoned commit	3	188	0	−0.102
Cartpole	Always-freeze	3	188	0	−0.117
Quadrotor	Clean REINFORCE	3	72	0	−0.030
Quadrotor	Poisoned action-only	3	72	68	−0.170
Quadrotor	Poisoned commit	3	72	0	−0.060
Quadrotor	Always-freeze	3	72	0	−0.029

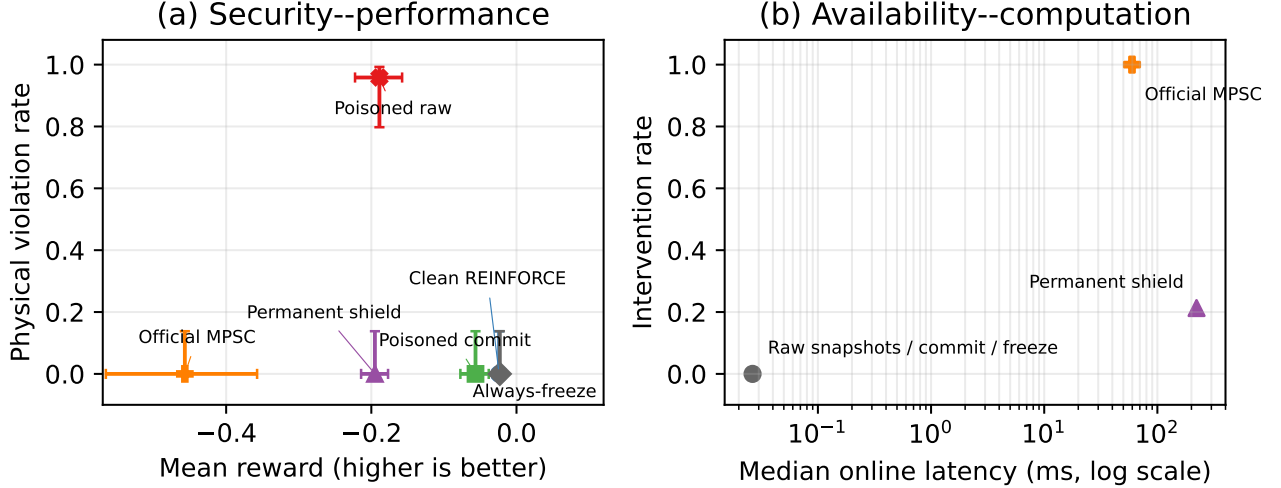


Fig. 2. The unified frontier on the same paired state blocks. Error bars in (a) are 95% Wilson intervals for violation rate and paired-block bootstrap intervals for mean rollout reward, conditional on the three locked learner snapshots. Panel (b) separates the negligible raw policy latency from the permanently-online protection cost; raw, commit, and freeze mechanisms overlap near 0.026 ms and zero intervention.

TABLE II

THREE-SEED PAIRED AUDIT AFTER INITIAL FIVE-STEP ACCEPTANCE. THE ONE-STEP KERNEL IS RETAINED AS A FAILED DEFENSE. A PREDICTIVE SWITCH IS A PERMANENT HANDOFF TO THE TRUSTED LQR BASELINE.

Contract	Failures	Actions	Empty kernel
Permanent raw release	19/71	0	0
One-step resident kernel	14/71	79	41
Predictive resident Simplex	0/71	19	0

backup shield, and Safe-Control-Gym’s official linear MPSC all have zero observed failures and complete 24/24. The shield intervenes on 511/2400 steps with no rejection, while MPSC intervenes on 2400/2400 with no infeasible solve. Median/p95 online latency is 221.925/238.216 ms for the shield and 59.657/76.055 ms for MPSC, compared with 0.026–0.027/0.028–0.029 ms for raw snapshot evaluation. These single-process wall-clock measurements are not worst-case execution-time guarantees.

Safety does not make these mechanisms utility-equivalent. Mean reward is −0.023 for freeze, −0.024 for clean, −0.056 for commit, −0.195 for the shield, and −0.457 for MPSC. Paired rollout deltas relative to freeze are

respectively −0.00096 (paired-block bootstrap 95% interval $[-0.00141, -0.00055]$), −0.03350 ($[-0.04754, -0.02182]$), −0.17208 ($[-0.18601, -0.15886]$), and −0.43393 ($[-0.54016, -0.33763]$); none improves on freeze in any of 24 blocks. This is a performance frontier under the locked attack evaluation, not a benign adaptation-utility experiment.

Table IV makes the snapshot-admission cost explicit. The common operating-set coverage is 99.3%, 100.0%, and 90.3% for learner seeds 2040–2042. Reconstructing trusted-state admission and the 21-point contiguous backtracking takes 8.49–9.32 s per seed and exactly reproduces the locked commit fractions 0.55, 0.40, and 0.70. Offline commit therefore avoids permanent per-action computation but pays finite certificate coverage and update latency. The comparison distinguishes three contracts: certify a snapshot before raw deployment, retain a filter permanently, or stop adaptation.

E. Trusted-State-Anchor Infrastructure

A trusted state anchor changes the state uncertainty justified by a certificate, but it does not sanitize the reward log used by our end-to-end attacker. We lock a separate sensitivity audit

TABLE III

UNIFIED PAIRED SECURITY-AVAILABILITY FRONTIER. WILSON 95% INTERVALS ARE $[0, 0.138]$ FOR EVERY 0/24 VIOLATION COUNT AND $[0.798, 0.993]$ FOR POISONED RAW. INTERVENTION IS A FRACTION OF EXECUTED STEPS. † DENOTES REWARD TRUNCATED AT THE FIRST SAFETY FAILURE AND THEREFORE NOT A UTILITY COMPARISON.

Mechanism	Viol./rollouts	Complete	Mean reward	Interv.	Latency med./p95 (ms)
Clean REINFORCE	0/24	24/24	-0.024	0.0%	0.027/0.029
Poisoned raw	23/24	1/24	-0.189†	0.0%	0.027/0.029
Poisoned commit	0/24	24/24	-0.056	0.0%	0.026/0.029
Always-freeze	0/24	24/24	-0.023	0.0%	0.026/0.028
Permanent shield	0/24	24/24	-0.195	21.3%	221.925/238.216
Official MPSC	0/24	24/24	-0.457	100.0%	59.657/76.055

TABLE IV

COMMIT AVAILABILITY AND OFFLINE COMPUTATION ON THE INDEPENDENT 144-STATE CANDIDATE POOL. COMMON ADMISSION REQUIRES BOTH FREEZE AND THE COMMITTED SNAPSHOT TO PASS THE 100-STEP, 0.003-GUARD CERTIFICATE.

Seed	Common/144	Commit frac.	Offline (s)
2040	143/144 (99.3%)	0.55	8.49
2041	144/144 (100.0%)	0.40	9.32
2042	130/144 (90.3%)	0.70	8.50

around that narrower contract. Altitude and pitch are anchored with bounded error; the remaining state channels stay trusted. Over the 12 update checkpoints, periods $P \in \{1, 3, 6, 12\}$ require 12, 4, 2, or 1 trusted calls. The final uncertainty radius is the high- or standard-quality base error plus a fixed growth per interval since the last anchor. Nested sampled plausible sets ensure that a less fresh condition contains all samples from fresher conditions.

Table V shows a bounded freshness effect. All conditions retain all 16 certificate centers and have 0/24 physical violations, compared with 23/24 for the poisoned raw snapshots and 0/24 for freeze on the same new blocks. Periods 1, 3, and 6 preserve the locked fractions 0.55/0.40/0.70. With only one anchor call ($P = 12$, final age 11), fractions become 0.50/0.40/0.65, reducing the pooled mean from 0.55 to 0.5167. The two quality grids produce identical fractions at the locked 0.05 backtracking resolution; we therefore claim no quality benefit.

The exact sensitivity propagation is vectorized over the largest nested set and costs 262.23, 276.11, and 262.51 s for the three learner seeds. Condition-specific trajectory counts range from 168,000 to 1,108,800; the shared runtime is an artifact-audit cost rather than an operational latency estimate for every condition. Anchor-conditioned reward also does not establish utility: relative to freeze, the paired mean delta is -0.01950 for $P \leq 6$ (paired-block bootstrap 95% interval approximately $[-0.0267, -0.0137]$) and -0.01445 for $P = 12$ ($[-0.01935, -0.01033]$), with zero improvements in 24 blocks. Sensor hardware latency, energy, authentication, and monetary cost are not measured; the defensible infrastructure metric here is the declared trusted-call count.

F. Boundary I: Official PPO

The residual-REINFORCE result does not establish learner-family universality. We therefore run a result-independent bounded audit using Safe-Control-Gym’s official pretrained PPO actor, critic, rollout buffer, clipped objective, and optimizer. The pretrained policy is safe on 72/72 held-out rollouts. A separately constructed normalized malicious target is unsafe on 72/72, so an unsafe policy exists inside the same action domain.

Under the locked reward-poison budget $L_\infty \leq 0.5$, the 16-batch checkpoint has 0/24 clean and 1/24 poisoned failures. At 24 batches both are 0/24, so the marginal failure does not persist. Neither the pre-specified physical condition nor the two-checkpoint margin condition passes, and the formal multi-seed sweep is not run. The poisoned updates do change target distance and task reward, but the evidence supports only update redirection, not a stable deep-PPO unsafe-policy attack.

G. Boundary II: Benign Adaptation Utility

Safety under attack does not by itself justify online learning. We pre-specify and lock a benign utility comparison between clean adaptation, always-freeze, and LifecycleGate on a trusted persistent differential quadrotor actuator bias. Calibration selects the smallest grid value that has zero state violations and decreases frozen-policy reward by at least 0.01: $b = 0.003$ N, with paired reward loss 0.01520.

In the single allowed development smoke, always-freeze obtains mean reward -0.03617, clean adaptation -0.04714, and LifecycleGate -0.08097. All three have 0/12 deployment violations. LifecycleGate has zero adaptation violations and filter rejections, accepts a nonzero update in 11/12 batches, and has mean accepted fraction 0.8208; nevertheless it improves zero of 12 paired rollouts over freeze. Its 12 finite-horizon certificate calls take 100.93 seconds. The pre-specified stop rule therefore fires and no formal three-seed utility experiment is run. This result rules out a tempting but invalid inference: parameter motion and certificate acceptance are not evidence of useful adaptation.

H. Measurement and Validity Boundaries

The commit and utility certificates use vectorized CasADi dynamics and fixed finite state sets; physical outcomes use independent PyBullet rollouts. They are sampled finite-horizon

TABLE V

TRUSTED-STATE-ANCHOR SENSITIVITY. RADIUS IS THE FINAL (z, θ) UNCERTAINTY IN M/RAD; SAMPLES ARE PER EACH OF 16 CERTIFICATE CENTERS. RETAINED FRACTIONS CORRESPOND TO LEARNER SEEDS 2040/2041/2042. EVERY ANCHOR-CONDITIONED SNAPSHOT IS DEPLOYED ON 24 IDENTICAL, NEW PAIRED PHYSICAL BLOCKS WITHOUT SNAPSHOT-SPECIFIC STATE PRE-ADMISSION.

Quality	P	Calls	Age	Radius (z, θ)	Samples	Retained 2040/41/42	Viol.
High	1	12	0	(0.001, 0.0005)	5	0.55/0.40/0.70	0/24
High	3	4	2	(0.005, 0.0025)	13	0.55/0.40/0.70	0/24
High	6	2	5	(0.011, 0.0055)	21	0.55/0.40/0.70	0/24
High	12	1	11	(0.023, 0.0115)	29	0.50/0.40/0.65	0/24
Standard	1	12	0	(0.005, 0.0020)	9	0.55/0.40/0.70	0/24
Standard	3	4	2	(0.009, 0.0040)	17	0.55/0.40/0.70	0/24
Standard	6	2	5	(0.015, 0.0070)	25	0.55/0.40/0.70	0/24
Standard	12	1	11	(0.027, 0.0130)	33	0.50/0.40/0.65	0/24

certificates, not continuous-state invariant-set proofs. Admission coverage, guard margins, model mismatch, and certificate latency must therefore be reported with violation counts.

We audit the physical action interface directly. For the benign actuator shift, the environment applies $u_{\text{actual}} = \text{clip}(u_{\text{command}} + [b, -b])$; the maximum observed implementation error is 2.61×10^{-11} N in the locked smoke. An initial calibration runner incorrectly counted Safe-Control-Gym’s aggregate constraint flag, which evaluates the disturbed request before physical clipping. This mislabeled actuator saturation as an applied-input violation. The audit correction changes only the outcome metric: realized state violations and pre-clip saturation are reported separately, while seeds, states, bias grid, and selection threshold remain unchanged.

I. Certificate Coverage and Model Mismatch

Reporting zero violations only on certificate-admitted states can be vacuous. We therefore lock a separate CasADi–PyBullet confusion audit before execution. It reconstructs the 12 clean, poisoned, commit, and freeze snapshots from the original traces, draws 144 states from three disjoint source seeds, performs no certificate admission, and compares horizons 20/50/100 and guard margins 0/0.001/0.003/0.005/0.01. The primary configuration is the pre-existing 100-step, 0.003-guard commit configuration; the grid is characterization rather than a selector.

Table VI evaluates 1728 snapshot–state pairs. CasADi certifies 1293; none violates the PyBullet state box within 100 steps. Clean, commit, and freeze coverage is 0.986, 0.944, and 1.000. The raw poisoned snapshots physically violate 403/432 pairs and have certificate coverage 0.062, so their low coverage is informative rejection rather than vacuous evidence for a safe deployed snapshot. Across all mechanisms the certificate conservatively rejects five physically safe pairs. Maximum action-interface error and actuator saturation are zero.

The grid exposes a model-mismatch boundary. Without a guard, poisoned snapshots produce 1, 2, and 1 false acceptances at horizons 20, 50, and 100. Every locked positive guard removes observed false acceptance, with increasing false rejection as the guard grows. Because 0.003 was declared as the primary pre-existing configuration before this audit, this is not post-hoc guard selection. Zero observed false acceptance

on this finite distribution still does not establish continuous-state invariance.

J. Artifact and Reproducibility Status

The artifact now includes analytic and tank frontiers, action- and parameter-level gates, adaptive attackers, paired cartpole and quadrotor learner audits, official PPO and MPSC adapters, and the pre-specified benign utility experiment, the common-state security–availability frontier, and the trusted-anchor grid. Every result above is written to row-level and aggregate CSV files. Generated table rows carry their raw source-file paths. The artifact also records exact commands, seed namespaces, the upstream Safe-Control-Gym revision, dependency versions, and SHA-256 checksums for the locked sources, raw results, derived tables and figures, upstream models, and submission PDF.

The security–availability costs are now aggregated on common paired state sets with separate offline and online timing, and trusted re-anchoring is characterized by explicit call counts, quality bounds, certificate computation, and physical outcomes. The coverage audit and its unguarded false-acceptance boundary must remain visible. These characterization tests do not authorize post-hoc changes to the failed PPO or utility protocols.

VIII. RELATED WORK AND DISCUSSION

Safety under severe sensor attacks. The closest sensor-attack baseline is the CBF-based severe sensor attack work of Tan et al. [7]. It considers an omniscient attacker that can spoof sensors, characterizes the set of plausible states when secure reconstruction is not unique, and designs offline and online safety filters. Our work does not replace that line of work. We use its core object—the plausible-state set and its feasible safety kernel—as the baseline on which a learning update must be gated.

Secure estimation and CPS attack limitations. Classical CPS security results characterize when attacks are detectable, identifiable, or correctable from system measurements [8], [11]. These works are foundational for the indistinguishability view used here: an attacked history can correspond to multiple physical states unless the sensing and dynamics provide

TABLE VI
CASADI-PYBULLET COVERAGE AT THE PRE-EXISTING 100-STEP, 0.003-GUARD CONFIGURATION. EACH ROW POOLS THREE LEARNER SNAPSHOTS OVER THE SAME 144 STATES.

Snapshot	Pairs	Physical viol.	Certified	False accept	False reject	Coverage
Clean REINFORCE	432	6	426	0	0	0.986
Poisoned action-only	432	403	27	0	2	0.062
Poisoned commit	432	21	408	0	3	0.944
Always-freeze	432	0	432	0	0	1.000

enough redundancy. Our contribution starts after that estimation boundary. Given the attacked-history state set that an estimator or safety filter can justify, we ask whether an online update of the controller can remain inside the corresponding safety certificate.

Runtime assurance and online control upgrades. The Simplex architecture was explicitly designed to introduce control-system upgrades online while a decision module retains an assured baseline controller [1]. It is therefore a direct lifecycle predecessor: monitoring and backup switching are not new claims of this work. Neural Simplex adds reverse switching and online retraining of a neural controller [2]. Barrier-based Simplex derives forward and reverse switching conditions from a barrier certificate and protects against an unsafe or adversarial advanced controller even with bounded dynamics error [3]. These architectures protect the deployed composition by retaining a runtime authority. Our question begins at a different contract transition: whether a finite reverse-switch check remains valid as authorization to remove that authority after adversarial update-log corruption. The direct resident-versus-release audit uses the same finite predictor on both sides of this transition.

Safety filters and shielded learning. Linear MPSC certifies or minimally modifies learning-based inputs by maintaining a recoverable path to a safe set [12]; adaptive MPSC shrinks a trusted parameter-uncertainty set and obtains nondecreasing feasible sets [13]. Safe-RL shields likewise forbid unsafe actions [14]. Under partial observability, Carr et al. [15] integrate a state estimator and shield and empirically study gradual or sudden shield removal after shielded training. These are important positive escape conditions. A permanent sound filter certifies $F \circ \pi$; removing it requires separate evidence that the learned raw snapshot is safe. Our focus is that latter admission obligation when update data are adversarial, rather than a claim that action shielding, partial-observability estimation, or runtime switching is new.

Shielded training followed by unprotected deployment is also not new as a deployment pattern. Hsu et al. [16] use supervisory shielding during exploration and PAC-Bayes evidence for transfer to unseen real environments. Carr et al. [15] empirically study gradual and sudden shield removal under partial observability. These works reinforce, rather than remove, our premise: release needs evidence whose scope matches the unprotected deployment contract. We study how a reward-log adversary invalidates a short one-time predictor while that same predictor remains effective when repeatedly

evaluated by resident authority.

Fault-tolerant CBF and safety-preserving architectures construct related positive mechanisms under specified faults, attacks, or bounded uncertainty [9], [17]. Online learning of barrier conditions can safely expand certifiable inputs under a bounded-error model [18], while guided exploration can collect data when robust filters become infeasible [19]. These results show how trusted learning and modeling assumptions can improve a certificate. Our threat model instead asks what remains sound when the learner’s update channel is itself corrupted.

Policy and certificate updates. Runtime-safety-guided repair uses interventions from a model-based safety controller to repair a learned policy and reduce later switching [4]. Repair with Preservation formally verifies already-correct initial-state regions while repairing additional ones [5]. Lyapunov-based safe policy learning projects each policy update or action into linearized feasible sets [6], and runtime monitoring can jointly repair neural controllers and certificate functions [20]. Most recently, adversarially robust conformal prediction transfers safety evidence across policy updates under interaction-driven distribution shift [21]. We do not claim that policy-update certification is absent from the literature. Our narrower contribution is an attack-specific reward-to-gate analysis and a paired counterexample to transferring resident reverse-switch evidence into permanent release.

Adversarial safety filters. ISAACS [22] and Gameplay Filters [23] synthesize robust safety filters using adversarial rollouts against disturbances, model error, or sim-to-real mismatch. These methods address how a fixed safety mechanism can intervene on task policies. Our setting differs because the attack corrupts the history used for online updates; the question is not only which action to filter, but whether the update that changes future actions can remain certified.

Policy and reward poisoning. Policy poisoning has been demonstrated for batch reinforcement learning and control [24]. Online attacks can perturb each reward under an ℓ_∞ budget and adapt to the learning process [25]; later work targets policy-gradient learners without known dynamics [26] and black-box deep-RL algorithms [27]. Reward poisoning itself is therefore not our novelty. We connect this attack channel to certificate inheritance in a physical controller: adaptation actions can remain protected while the raw deployable snapshot fails later, and snapshot admission, freeze, and permanent filtering expose different physical, coverage, and availability outcomes.

TABLE VII
CLOSEST-WORK BOUNDARY. THE TABLE DISTINGUISHES SECURITY QUESTIONS; IT DOES NOT RANK THE STRENGTH OF INCOMPARABLE SAFETY GUARANTEES.

Line of work	Update/advanced-controller setting	Authority after evidence	Primary contract
Neural and barrier Simplex [2], [3]	Online retraining or unsafe/adversarial advanced controller	Decision module and baseline remain resident	Safe composition with forward/reverse switching
Policy repair and safe updates [4]–[6]	Repair, projection, or preservation is the defense	Runtime controller or verified/projected policy	Reduce intervention or preserve verified behavior
Shielded transfer [15], [16]	Shielded training; update log not the adversarial channel	Shield may be removed under separate transfer evidence	Generalization or empirical post-shield safety
This work	Bounded reward-log adversary during protected adaptation	Resident authority and permanent removal are paired	Can finite reverse-switch evidence authorize raw release?

Certified robustness and continual learning. Certified safety for reinforcement learning under adversarial input perturbations studies safety properties under perturbed observations [28]. Certified continual learning studies how to preserve verified properties across neural-network retraining [29]. Our problem combines these concerns in a closed-loop CPS setting: the property is a physical safety certificate, and the retraining data can be strategically manipulated by FDI.

Adaptive control under attacks. Model-free adaptive predictive control has been studied under deception and DoS attacks. Li and Hou [30] model feedback-channel deception with multiplicative and additive attack factors and prove mean-square boundedness of the tracking error. Hou et al. [31] reconstruct hijacked packets in wind-power microgrids and derive attack-duration and attack-frequency conditions for frequency recovery and stability. These results show that adaptive controllers can be designed for attacked networked control settings. They do not formulate a physical safety certificate over an FDI-induced plausible-state set, nor do they constrain the update that changes future certified actions. This distinction is important: our claim is not that adaptive control under attack is new, but that certified online updates require an explicit kernel gate.

Discussion. A permanent sound filter can protect the composition $F \circ \pi$, and our resident predictive and MPSC results confirm this escape condition. It does not certify a raw updated snapshot after the filter is removed. Conversely, the failed one-step kernel shows that residence alone is not sufficient: the decision rule must preserve recoverability. Snapshot admission, permanent predictive authority, action filtering, always-freeze, and trusted re-anchoring are different assurance contracts.

The evidence has seven important limitations. First, the necessary gate and conditional finite-abstraction lift are not complete controller-synthesis results: the lift assumes a justified joint reachable-set cover and known regularity/model-error bounds. Second, the implemented parameter and commit gates use finite future-history or state samples without such a cover proof, so their soundness is limited by abstraction, model mismatch, horizon, and guard margin. Third, the five-step contract horizon was developed on learner seed 2040 before being frozen for the disjoint-state, three-seed audit; it is not pristine confirmatory evidence. Fourth, the gate-aware target analysis is retrospective: the original poison was not

optimized under that newly written constraint. Fifth, the exact reward-influence theorem covers only batches where gradient and parameter clipping remain inactive; the clipped quadrotor attack is empirical. Sixth, the end-to-end physical attack is established for low-dimensional residual REINFORCE; the official-PPO audit does not show a stable physical failure, and the benign-shift smoke does not show utility over always-freeze. Seventh, the anchor audit models bounded error on only altitude and pitch and reports calls rather than platform-specific sensing latency, energy, or authentication cost. These are result boundaries, not omitted positive evidence.

IX. CONCLUSION

Protected adaptation and safe raw release are different security contracts. A resident decision module can repeatedly detect that an evolving raw policy is leaving a recoverable region; a one-time pass of the same finite predictor does not cover the unmonitored future after permanent release. Under bounded reward-log corruption, our three-seed paired cartpole audit makes this distinction concrete: raw release fails on 19/71 initially accepted pairs, whereas repeated predictive authority switches on exactly those pairs and has no observed failure. A one-step resident action kernel still fails on 14/71, so the result depends on recoverability, not residence alone.

The normalized reward-influence analysis explains how a reward-only attacker can move a standardized-advantage update toward release-gate halfspaces. Its exact SOCP support result is batch-local and excludes active gradient or parameter clipping. The broader cartpole and quadrotor experiments, finite-horizon commit gate, freeze, backup shield, and official MPSC then expose the coverage, intervention, reward, and latency costs of different escape contracts.

The negative boundaries are part of the result. The release horizon was development-fixed rather than pristine, the gate-aware target analysis is retrospective, the physical attack is limited to low-dimensional residual REINFORCE, the fixed-budget effect does not stably transfer to official PPO, and admitted updates do not beat freeze in the benign-adaptation smoke. Accordingly, we claim neither a new Simplex architecture nor a universal learning impossibility. The defensible conclusion is narrower: evidence generated under resident runtime authority is contract-bound, and permanent raw release

after adversarial adaptation requires independent evidence that covers the future from which that authority has been removed.

REFERENCES

- [1] D. Seto, B. Krogh, L. Sha, and A. Chutinan, "The simplex architecture for safe on-line control system upgrades," in *Proceedings of the 1998 American Control Conference*, vol. 6, 1998, pp. 3504–3508.
- [2] D. T. Phan, R. Grosu, N. Jansen, N. Paoletti, S. A. Smolka, and S. D. Stoller, "Neural simplex architecture," in *NASA Formal Methods*, ser. Lecture Notes in Computer Science, vol. 12229. Springer, 2020, pp. 97–114.
- [3] A. Damare, S. Roy, R. Sharma, K. DSouza, S. A. Smolka, and S. D. Stoller, "A barrier certificate-based simplex architecture for systems with approximate and hybrid dynamics," *IEEE Access*, vol. 13, pp. 157 742–157 763, 2025.
- [4] W. Zhou, R. Gao, B. Kim, E. Kang, and W. Li, "Runtime-safety-guided policy repair," in *Runtime Verification*, ser. Lecture Notes in Computer Science, vol. 12399. Springer, 2020, pp. 131–150.
- [5] P. Lu, M. Cleaveland, O. Sokolsky, I. Lee, and I. Ruchkin, "Repairing learning-enabled controllers while preserving what works," in *2024 ACM/IEEE 15th International Conference on Cyber-Physical Systems*. IEEE, 2024, pp. 1–11.
- [6] Y. Chow, O. Nachum, A. Faust, E. Dueñez-Guzman, and M. Ghavamzadeh, "Safe policy learning for continuous control," in *Proceedings of the 2020 Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 155. PMLR, 2021, pp. 801–821. [Online]. Available: <https://proceedings.mlr.press/v155/chow21a.html>
- [7] X. Tan, P. Ong, P. Tabuada, and A. D. Ames, "Safety of linear systems under severe sensor attacks," 2024. [Online]. Available: <https://arxiv.org/abs/2409.08413>
- [8] H. Fawzi, P. Tabuada, and S. Diggavi, "Secure estimation and control for cyber-physical systems under adversarial attacks," *IEEE Transactions on Automatic Control*, vol. 59, no. 6, pp. 1454–1467, 2014.
- [9] H. Zhang, Z. Li, and A. Clark, "Safe control for nonlinear systems under faults and attacks via control barrier functions," 2022. [Online]. Available: <https://arxiv.org/abs/2207.05146>
- [10] Z. Yuan, A. W. Hall, S. Zhou, L. Brunke, M. Greeff, J. Panerati, and A. P. Schoellig, "Safe-control-gym: A unified benchmark suite for safe learning-based control and reinforcement learning in robotics," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 11 142–11 149, 2022.
- [11] F. Pasqualetti, F. Dörfler, and F. Bullo, "Attack detection and identification in cyber-physical systems," *IEEE Transactions on Automatic Control*, vol. 58, no. 11, pp. 2715–2729, 2013.
- [12] K. P. Wabersich and M. N. Zeilinger, "Linear model predictive safety certification for learning-based control," in *2018 IEEE Conference on Decision and Control*, 2018, pp. 7130–7135.
- [13] A. Didier, K. P. Wabersich, and M. N. Zeilinger, "Adaptive model predictive safety certification for learning-based control," in *2021 60th IEEE Conference on Decision and Control*, 2021, pp. 809–815.
- [14] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, "Safe reinforcement learning via shielding," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 32, no. 1, 2018.
- [15] S. Carr, N. Jansen, S. Junges, and U. Topcu, "Safe reinforcement learning via shielding under partial observability," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 12, 2023, pp. 14 748–14 756.
- [16] K.-C. Hsu, A. Z. Ren, D. P. Nguyen, A. Majumdar, and J. F. Fisac, "Sim-to-lab-to-real: Safe reinforcement learning with shielding and generalization guarantees," *Artificial Intelligence*, p. 103811, 2023.
- [17] M. Attar and W. Lucia, "A data-driven safety preserving control architecture for constrained cyber-physical systems," 2023. [Online]. Available: <https://arxiv.org/abs/2312.00658>
- [18] L. Brunke, S. Zhou, and A. P. Schoellig, "Barrier bayesian linear regression: Online learning of control barrier conditions for safety-critical control of uncertain systems," in *Proceedings of the 4th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 168. PMLR, 2022, pp. 881–892. [Online]. Available: <https://proceedings.mlr.press/v168/brunke22a.html>
- [19] A. Capone, R. K. Cosner, A. Ames, and S. Hirche, "Learning safe control via on-the-fly bandit exploration," in *Proceedings of the 42nd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 6726–6745. [Online]. Available: <https://proceedings.mlr.press/v267/capone25a.html>
- [20] E. Yu, D. Žikelić, and T. A. Henzinger, "Neural control and certificate repair via runtime monitoring," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 25, 2025, pp. 26 409–26 417.
- [21] O. Mirzaeododangeh, E. S. Shekhtman, N. Matni, and L. Lindemann, "Safe planning in interactive environments via iterative policy updates and adversarially robust conformal prediction," in *Proceedings of The 8th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 331. PMLR, 2026, pp. 678–704. [Online]. Available: <https://proceedings.mlr.press/v331/mirzaeododangeh26a.html>
- [22] K.-C. Hsu, D. P. Nguyen, and J. F. Fisac, "ISAACS: Iterative soft adversarial actor-critic for safety," in *Proceedings of The 5th Annual Learning for Dynamics and Control Conference*, ser. Proceedings of Machine Learning Research, vol. 211. PMLR, 2023, pp. 90–103. [Online]. Available: <https://proceedings.mlr.press/v211/hsu23a.html>
- [23] D. P. Nguyen, K.-C. Hsu, W. Yu, J. Tan, and J. F. Fisac, "Gameplay filters: Safe robot walking through adversarial imagination," 2024. [Online]. Available: <https://arxiv.org/abs/2405.00846>
- [24] Y. Ma, X. Zhang, W. Sun, and X. Zhu, "Policy poisoning in batch reinforcement learning and control," in *Advances in Neural Information Processing Systems*, vol. 32, 2019, pp. 14 543–14 553.
- [25] X. Zhang, Y. Ma, A. Singla, and X. Zhu, "Adaptive reward-poisoning attacks against reinforcement learning," in *Proceedings of the 37th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 119. PMLR, 2020, pp. 11 225–11 234. [Online]. Available: <https://proceedings.mlr.press/v119/zhang20u.html>
- [26] Y. Sun, D. Huo, and F. Huang, "Vulnerability-aware poisoning mechanism for online RL with unknown dynamics," in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://openreview.net/forum?id=9r30XCjf5Dt>
- [27] Y. Xu, Q. Zeng, and G. Singh, "Efficient reward poisoning attacks on online deep reinforcement learning," *Transactions on Machine Learning Research*, 2025. [Online]. Available: <https://openreview.net/forum?id=25G63IDHV2>
- [28] J. Wu, H. Sibai, and Y. Vorobeychik, "Certifying safety in reinforcement learning under adversarial perturbation attacks," 2022. [Online]. Available: <https://arxiv.org/abs/2212.14115>
- [29] L. H. Pham and J. Sun, "Certified continual learning for neural network regression," 2024. [Online]. Available: <https://arxiv.org/abs/2407.06697>
- [30] F. Li and Z. Hou, "Event-triggered model-free adaptive predictive control for networked control systems under deception attacks," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 54, no. 2, pp. 1325–1334, 2024.
- [31] R. Hou, L. Jia, X. Bu, and J. Li, "Event-triggered model-free adaptive predictive control for networked wind-power microgrids subject to aperiodic DoS attacks," *IEEE Transactions on Information Forensics and Security*, vol. 21, pp. 1737–1750, 2026.